Experiment 07: IoT Control Using MQTT Broker

Roll Number: 21BIT191

Objective: IoT Control using MQTT

Equipment and Components: Raspberry Pi – 4B, SD card/ USB Drive, Laptop, SD card reader (optional), jumper wires, LED, breadbroad

Theory: This experiment "IoT Control using MQTT Broker" aims to achieve remote control of an LED connected to a Raspberry Pi using a cloud-based MQTT broker. MQTT, a lightweight messaging protocol tailored for IoT, uses a publish-subscribe model where devices communicate through a central broker. The hardware setup involves connecting an LED to the Raspberry Pi, while the software setup requires Raspbian OS, Python, the paho-mqtt library, and an MQTT broker (the public test broker from Eclipse Mosquitto is used). The provided Python script enables control of the LED through MQTT messages, which can be published either via the Mosquitto command line or another Python script. The LED responds by turning on or off based on the received "ON" or "OFF" messages, showcasing the power of MQTT in IoT control applications.

Connection of the LED with the Raspberry Pi:

LED (Anode +):

- The positive leg (longer leg) of the LED is connected to **GPIO 4** on the Raspberry Pi. GPIO pins are General Purpose Input/Output pins that can be controlled by software to provide digital signals (HIGH or LOW).
- **LED (Cathode -):**
- The negative leg (shorter leg) of the LED is connected to **GND** (Ground) on the Raspberry Pi. This provides the return path for the current flowing through the LED.
-

Python Script for Reading Data

The provided Python script facilitates the control of an LED connected to a Raspberry Pi via an MQTT broker. It establishes a connection to the MQTT broker, subscribes to a specific topic, and sets up a callback function to handle incoming messages. When a message is received on the subscribed topic, the script decodes the payload and checks if it's "ON" or "OFF". Based on the message, it controls the LED connected to the specified GPIO pin on the Raspberry Pi, turning it on or off accordingly. This enables remote control of the LED through MQTT messages published to the designated topic.

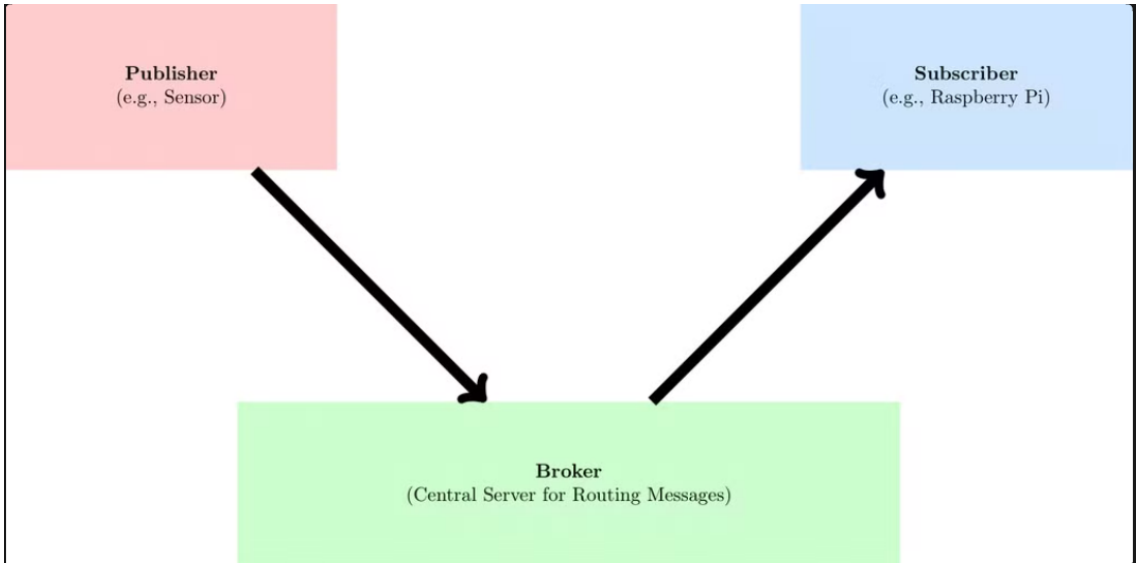
MQTT Concepts:

Lightweight Protocol: MQTT is designed to be efficient, especially for resource-constrained IoT devices and networks with limited bandwidth or unreliable connectivity.

- **Publish-Subscribe Model:** This model decouples communication between devices.
  - Publishers send messages to specific topics.
  - Subscribers express interest in receiving messages on particular topics.
  - The broker acts as the intermediary, ensuring messages reach the intended subscribers.

MQTT Architecture and Communication Flow:

- **Central Broker:** The broker is the core of the MQTT architecture. It receives messages from publishers and distributes them to the appropriate subscribers based on topic subscriptions.
- **Publishers & Subscribers:** Multiple devices (clients) can connect to the broker. Some act as publishers, sending data, while others act as subscribers, receiving data.
- **Topic-Based Routing:** Topics are used to categorize messages. This allows for flexible and efficient message delivery as subscribers only receive messages relevant to their interests.



Example Scenario:

- A temperature sensor (publisher) sends a message like "Temperature: 25°C" to the topic "home/room/temperature".
- A Raspberry Pi (subscriber) has subscribed to the "home/room/temperature" topic.
- The broker receives the temperature message and forwards it to the Raspberry Pi.

Procedure:

1. After setting up raspberry pi in experiment 2, now you can direct open PuTTY and repeat same steps as done previously. So, the Raspberry Pi OS desktop will appear.

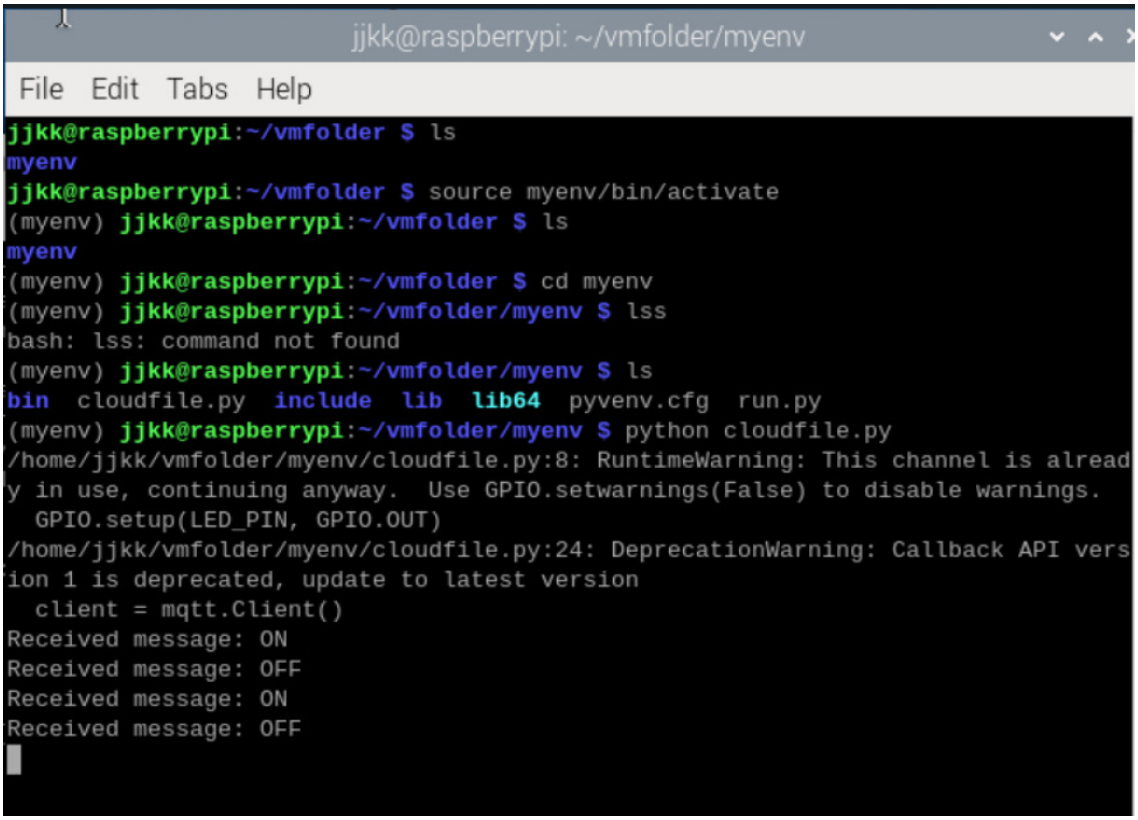
2. Now, we need to update our Raspberry Pi.Ensure that you have the latest software and libraries:
   We need to install the Mosquitto Clients:
   Open the terminal of the Raspberry Pi and type: sudo apt install mosquitto mosquitto-clients
   After installation, you'll be able to use mosquitto_pub and mosquitto_sub to publish and subscribe to MQTT messages via the terminal.
   Now also install the paho-mqtt library into you activated virtual environment
   pip install paho-mqtt
3. Now we need to create a python script having name cloudfile.py
   Write the python code given below

```python
import RPi.GPIO as GPIO
import paho.mqtt.client as mqtt

# GPIO Setup
LED_PIN = 4
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

# Callback when a message is received
def on_message(client, userdata, message):
    payload = message.payload.decode()
    print(f"Received message: {payload}")
    if payload == "ON":
        GPIO.output(LED_PIN, GPIO.HIGH)
    elif payload == "OFF":
        GPIO.output(LED_PIN, GPIO.LOW)

# MQTT Setup
broker_url = "test.mosquitto.org"  # Eclipse Mosquitto public broker
broker_port = 1883  # Default unencrypted MQTT port
topic = "jjkk/mqtt/on/off"

client = mqtt.Client()
client.connect(broker_url, broker_port)
client.subscribe(topic)

client.on_message = on_message

# Start MQTT client loop
client.loop_start()

try:
    while True:
        pass  # Keep the script running
except KeyboardInterrupt:
    print("Experiment stopped")
finally:
    GPIO.cleanup()
    client.loop_stop()
```

4. Now make a second python file having name as run.py

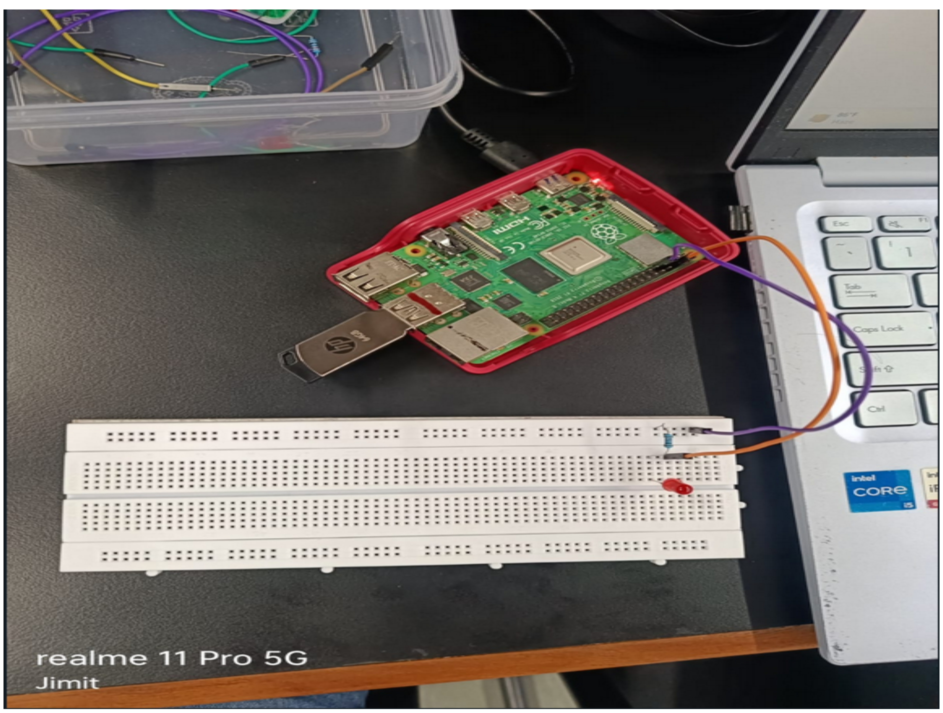
```python
import paho.mqtt.client as mqtt

broker_url = "test.mosquitto.org"
broker_port = 1883
topic = "jjkk/mqtt/on/off"

client = mqtt.Client()
client.connect(broker_url, broker_port)

# Publish message to turn the LED ON
#client.publish(topic, "ON")

# Or to turn the LED OFF
client.publish(topic, "OFF")
```

Result:



```
jjkk@raspberrypi: ~/vmfolder/myenv
File  Edit  Tabs  Help
jjkk@raspberrypi:~/vmfolder $ source myenv/bin/activate
(myenv) jjkk@raspberrypi:~/vmfolder $ pip install paho-mqtt
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Requirement already satisfied: paho-mqtt in ./myenv/lib/python3.11/site-packages
 (2.1.0)
(myenv) jjkk@raspberrypi:~/vmfolder $ python run.py
python: can't open file '/home/jjkk/vmfolder/run.py': [Errno 2] No such file or
directory
(myenv) jjkk@raspberrypi:~/vmfolder $ ls
myenv
(myenv) jjkk@raspberrypi:~/vmfolder $ cd myenv
(myenv) jjkk@raspberrypi:~/vmfolder/myenv $ python run.py
/home/jjkk/vmfolder/myenv/run.py:7: DeprecationWarning: Callback API version 1 i
s deprecated, update to latest version
  client = mqtt.Client()
(myenv) jjkk@raspberrypi:~/vmfolder/myenv $ python run.py
/home/jjkk/vmfolder/myenv/run.py:7: DeprecationWarning: Callback API version 1 i
s deprecated, update to latest version
  client = mqtt.Client()
(myenv) jjkk@raspberrypi:~/vmfolder/myenv $ python run.py
/home/jjkk/vmfolder/myenv/run.py:7: DeprecationWarning: Callback API version 1 i
s deprecated, update to latest version
  client = mqtt.Client()
(myenv) jjkk@raspberrypi:~/vmfolder/myenv $
```

Connection Diagram:



Discussion: The experiment successfully demonstrates the implementation of IoT control using MQTT. By leveraging a cloud-based MQTT broker and a simple Python script, we were able to remotely control an LED connected to a Raspberry Pi. The publish-subscribe model of MQTT allowed for efficient communication between the Raspberry Pi (subscriber) and the MQTT broker, enabling seamless control of the LED based on messages published to the designated topic. The experiment highlights the flexibility and potential of MQTT in IoT applications, where devices with varying capabilities can interact and exchange data in a scalable and reliable manner.

Conclusion: This lab exercise provided hands-on experience with MQTT and its application in IoT control scenarios. We successfully established communication between a Raspberry Pi and an MQTT broker, allowing us to control an LED remotely. The experiment underscores the significance of MQTT as a lightweight and efficient protocol for IoT communication. It also showcases the ease with which devices can be integrated into an IoT ecosystem using MQTT, paving the way for a wide range of innovative and interconnected applications in the future.